Phần 1: Component, Props & State (Cơ bản)

Bài 1: Thẻ thông tin thành viên (UserCard)

Mục tiêu: Biết định nghĩa kiểu dữ liệu cơ bản cho Props của một Functional Component.

Bài tập: Tạo component UserCard nhận vào name (string), age (number), và isAdmin (boolean) để hiển thị giao diện khác nhau cho Admin.

Kiến thức cần học: interface, primitive types (string, number, boolean).

Bài 2: Bộ đếm số tăng giảm (Counter)

Mục tiêu: Hiểu cách TypeScript tự động suy luận kiểu (Type Inference) với React State.

Bài tập: Tạo component Counter có 2 nút tăng/giảm giá trị. Giới hạn không cho giảm xuống dưới 0.

Kiến thức cần học: useState với kiểu dữ liệu ngầm định và tường minh (useState<number>).

Bài 3: Danh sách việc cần làm đơn giản (Todo Basic)

Mục tiêu: Quản lý State dạng mảng chứa các object có cấu trúc.

Bài tập: Tạo màn hình TodoList chứa mảng các task. Mỗi task gồm id (string) và title (string). Hiển thị ra màn hình bằng map().

Kiến thức cần học: Định nghĩa mảng object (Todo[] hoặc Array<Todo>).

Bài 4: Nút bấm tùy biến (Custom Button)

Mục tiêu: Định nghĩa kiểu cho các thuộc tính tùy chọn (Optional Props) và hàm callback.

Bài tập: Tạo component MyButton nhận vào label, hàm onPress, và một thuộc tính color không bắt buộc (nếu không truyền thì mặc định là màu xanh).

Kiến thức cần học: Optional property (?), định nghĩa kiểu cho hàm (() => void).

Bài 5: Ô nhập text phản hồi (Live Input)

Mục tiêu: Xác định kiểu cho dữ liệu nhận được từ sự kiện thay đổi text.

Bài tập: Tạo ô nhập TextInput hiển thị số lượng ký tự người dùng đang gõ theo thời gian thực.

Kiến thức cần học: Kiểu dữ liệu chuỗi, thuộc tính length của string trong TS.

Bài 6: Trạng thái tải trang (Loading Wrapper)

Mục tiêu: Sử dụng kiểu dữ liệu ReactNode để truyền component con vào component cha.

Bài tập: Tạo component LoadingContainer nhận vào isLoading (boolean) và children. Nếu đang load thì hiện ActivityIndicator, ngược lại hiện children.

Kiến thức cần học: React.ReactNode hoặc PropsWithChildren.

Bài 7: Chọn giới tính (Gender Selector)

Mục tiêu: Giới hạn dữ liệu nhập vào bằng kiểu Union thay vì dùng chuỗi tự do.

Bài tập: Tạo component chọn giới tính. State gender chỉ được phép nhận một trong ba giá trị: 'male', 'female', hoặc 'other'.

Kiến thức cần học: Union Types ('male' | 'female' | 'other').

Bài 8: Component tái sử dụng với Props & Variant (Compound Props Pattern)

Mục tiêu: Rèn kỹ năng thiết kế API component linh hoạt, tách style theo cấu hình.

Bài tập: Xây dựng component `AppButton` dùng chung cho toàn bộ app BookStore, hỗ trợ nhiều biến thể (variant) khác nhau.

- Props: `title`, `variant` (`primary` | `secondary` | `outline` | `danger`), `size` (`small` | `medium` | `large`), `onPress`, `disabled`, `loading`, `icon` (component tùy chọn).
- Khi `loading = true`, hiển thị `ActivityIndicator` thay cho `title`.
- Style của từng `variant`/`size` phải tách riêng bằng object style động (không viết if-else lồng trong JSX).
- Áp dụng thử cho các nút: "Thêm vào giỏ hàng" (primary), "Xem chi tiết" (outline), "Xóa sách" (danger).

Bài 9: Higher-Order Component (HOC) cho xử lý trạng thái tải dữ liệu

Yêu cầu: Viết HOC `withLoadingState(WrappedComponent)` áp dụng cho các màn hình danh sách sách.

- HOC nhận thêm props `isLoading`, `isError`, `errorMessage`.
- Khi `isLoading = true` → hiển thị `ActivityIndicator` toàn màn hình.
- Khi `isError = true` → hiển thị component `ErrorView` kèm nút "Thử lại".
- Ngược lại → render `WrappedComponent` với các props còn lại (`...props`).
- Áp dụng HOC cho màn hình `BookListScreen` và `OrderHistoryScreen`.

Mục tiêu: Hiểu nguyên lý HOC, tái sử dụng logic UI trạng thái (loading/error) giữa nhiều màn hình.

Bài 10: Compound Component Pattern

Yêu cầu: Xây dựng bộ component `BookCard` theo mô hình *compound component* (component cha quản lý state, các component con giao tiếp ngầm qua Context).

```
<BookCard book={item}>
  <BookCard.Cover />
  <BookCard.Title />
  <BookCard.Price />
  <BookCard.FavoriteButton />
</BookCard>
```

- `BookCard` tạo Context chứa dữ liệu `book` và state `isFavorite`.
- Các component con (`Cover`, `Title`, `Price`, `FavoriteButton`) lấy dữ liệu qua `useContext`, không nhận props trực tiếp từ ngoài.
- `FavoriteButton` khi bấm sẽ toggle `isFavorite` và đổi icon trái tim.

Mục tiêu: Hiểu cách chia nhỏ component phức tạp nhưng vẫn giữ API gọi gọn gàng, làm quen Context API trong component design.